



mlFlow

Contexto

- Primer entrenamiento: 8 Enero 2024: Xgboost con AUC 80
- Segundo entrenamiento: 8 Junio 2024: con AUC 79
- ¿Ha mejorado el train? ¿ha empeorado?
- ¿Donde está registrada la trazabilidad del experimento?

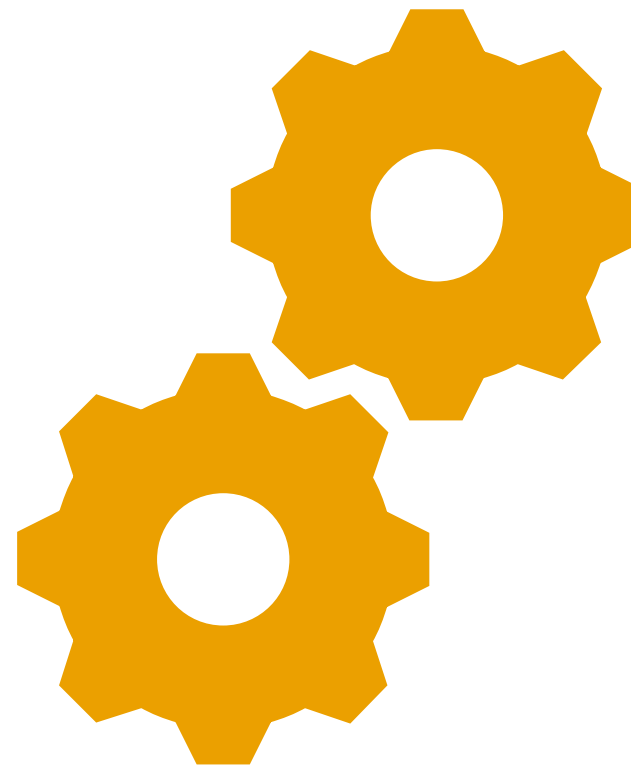
Qué vas a conseguir ?

Histórico de entrenamientos (parámetros, métricas, artefactos, modelos) con MLflow Tracking



Contexto

En la empresa convivimos con decenas de modelos de Machine Learning desplegados en distintos productos y procesos. Estos modelos no se comportan igual: algunos requieren reentrenamiento **semanal** por cambios rápidos en los datos, otros se actualizan **mensualmente** y otros solo cada **seis o nueve meses** porque su dominio es más estable o porque dependen de ciclos de negocio. Además, los datasets, las features y los hiperparámetros evolucionan con el tiempo, y diferentes equipos contribuyen a los pipelines.



Contexto

En este contexto, el principal reto no es únicamente “entrenar modelos”, sino mantener trazabilidad y control sobre el histórico de entrenamientos:

- qué datos se usaron (periodo, versión, filtros),
- con qué código y dependencias,
- qué configuración (features, hiperparámetros, seed),
- qué métricas y validaciones se obtuvieron,
- qué artefactos se generaron (modelo, scaler, encoder, gráficos),
- y qué versión se promovió a staging/producción.

Contexto

Sin una solución estándar, se acaba trabajando con logs dispersos, nombres de ficheros inconsistentes, notebooks no reproducibles y un “estado del modelo” difícil de auditar. Esto complica la operación: cuando un modelo cae de rendimiento o cambia el comportamiento del negocio, es costoso responder preguntas básicas **como “¿cuál fue el último entrenamiento bueno?”, “¿qué cambió respecto a la versión anterior?” o “¿podemos reproducir exactamente ese resultado?”**.

Contexto

Por eso proponemos MLflow como capa transversal de MLOps para centralizar:

1. Tracking de experimentos y reentrenos: cada entrenamiento queda registrado como un run con métricas, parámetros y artefactos.
2. Versionado de modelos: registro de versiones y promoción controlada (dev → staging → prod).
3. Reproducibilidad: captura de entorno, dependencias, commit y configuración del pipeline.
4. Gobernanza y auditoría: histórico completo, comparables entre modelos/equipos, con permisos y trazabilidad.
5. Operación con múltiples cadencias: el mismo estándar sirve tanto para reentrenos semanales como para actualizaciones semestrales, manteniendo un historial consistente a lo largo de años.

Contexto

El objetivo final es convertir el entrenamiento en un proceso observable y gobernable, donde cada modelo tenga un “libro de vida” completo: qué se entrenó, cuándo, por qué, con qué datos y con qué resultado, facilitando mantenimiento, debugging, compliance y toma de decisiones basada en evidencia.

Flavors

MLflow no “soporta modelos” por algoritmo (árbol, red, etc.) sino por integraciones llamadas flavors: si tu modelo pertenece a una librería soportada, MLflow lo puede loggear, versionar, cargar y servir siguiendo ese flavor.

ML clásico / GBMs

- sklearn (scikit-learn)
- Xgboost
- lightgbm
- Catboo
- ststatsmodels



Flavors



Deep Learning

- Pytorch
- Ensorflow
- transformers (modelos de Hugging Face, incluyendo interfaces tipo chat/completions)

Spark / Big Data

- spark y pyspark.ml (para pipelines y modelos en Apache Spark)

Flavors

NLP

- Spacy

Time series

- Prophet
- Pmdarima

Interoperabilidad

- onnx (export/uso en formato ONNX)



Flavors

LLMOps / apps LLM

- openai (integración con OpenAI)
- Angchain
- llama_indexdspy

Otros frameworks

- H2o
- paddle

